

Practical No: 10

Aim:

To create and apply validation rules on MongoDB collections.

Theory:

MongoDB is traditionally known as a **schema-less** or **schema-flexible** NoSQL database, meaning it does not require a rigid, predefined structure for documents. This flexibility is beneficial for dynamic applications, evolving data models, and rapid development. However, in many real-world systems such as financial applications, inventory systems, or academic databases, certain rules and constraints must be maintained to ensure data accuracy and consistency.

A schema validation rule typically defines:

- **bsonType**: Expected datatype (e.g., string, number, object, array)
- **required**: List of mandatory fields
- **properties**: Constraints for each field, including type and other validation rules

Schema validation provides several benefits:

1. **Data Integrity** – Ensures that stored data adheres to expected formats.
2. **Error Prevention** – Catch mistakes early before they reach application logic.
3. **Standardization** – Makes collections behave more like structured relational tables.
4. **Security & Reliability** – Prevents malicious or malformed data from being stored.

In this experiment, a collection named `books` is created with a validator enforcing a JSON schema. A valid document is successfully inserted, while an invalid document containing a string instead of a number for the `price` field triggers an error. This demonstrates how MongoDB ensures structured data without compromising flexibility.

Commands:

```
db.createCollection("books", { validator: { $jsonSchema: { bsonType: "object", required: ["title", "price"], properties: { title: { bsonType: "string" }, price: { bsonType: "number" } } } } })
db.books.insertOne({ title: "AI Book", price: 500 }) // valid
db.books.insertOne({ title: "Wrong Book", price: "500" }) // ERROR

db.createCollection("students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "age", "dept"],
      properties: {
        name: {
          bsonType: "string",
          description: "Name must be a string and is required"
```

```

    },
    age: {
      bsonType: "int",
      minimum: 18,
      maximum: 30,
      description: "Age must be an integer between 18 and 30"
    },
    dept: {
      bsonType: "string",
      description: "Dept must be a string and is required"
    }
  }
},
validationLevel: "strict",
validationAction: "error"
})

```

Explanation

- **validator**: Defines the rules
- **\$jsonSchema**: Uses JSON Schema for validation
- **validationLevel: strict**: All inserts and updates are validated
- **validationAction: error**: Reject invalid documents

Update Existing Collection to Add Validation

```

db.runCommand({
  collMod: "students",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "age", "dept"],
      properties: {
        name: { bsonType: "string" },
        age: { bsonType: "int", minimum: 18, maximum: 30 },
        dept: { bsonType: "string" }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
})

```

Test Validation

Valid Insert

```
db.students.insertOne({ name: "Rahul", age: 22, dept: "IT" })
```

✓ Works fine.

Invalid Insert

```
db.students.insertOne({ name: "Sneha", age: 17, dept: "Computer" })
```

✗ Fails because **age < 18**.

Expected Output:

Invalid documents are rejected with a validation error.

Viva Questions:

1. What is schema validation in MongoDB?

Schema validation is a mechanism that enforces rules on documents to ensure they follow a predefined structure and data type before insertion or update.

2. Which option is used to apply validation rules while creating a collection?

The validator option is used during collection creation.

3. What is \$jsonSchema in MongoDB?

\$jsonSchema is a standard used to define validation rules such as required fields, data types, and structure of documents.

4. What does bsonType specify?

bsonType specifies the expected data type of a field, such as string, number, object, or array.

5. What happens when a document violates validation rules?

MongoDB rejects the document and throws a validation error.

Conclusion: In this experiment, schema validation was successfully implemented in MongoDB using the \$jsonSchema validator. The books collection was created with predefined structural

and datatype rules. A valid document was inserted successfully, while an invalid document was rejected, demonstrating how MongoDB enforces data integrity.